Wallets & Ledger — Table Layout & Balance Update Flow

InstantMudra Enterprises Admin Portal

32 physical wallet tables — no shared table, one pair per level × wallet type

AD	ET	WL	SD	PT	DT	RT
Prepaid + Postpaid	Prepaid + Postpaid	Prepaid + Postpaid	Prepaid + Postpaid	Prepaid + Postpaid	+ Gullak	+ Gullak

7 levels × 2 wallet types (Prepaid, Postpaid) = 14 balance tables, + Gullak only for DT & RT (+2) = 16 balance tables
— each mirrored by a matching *TransactionDetails ledger table = **32 physical tables total**, no shared table

Two @MappedSuperclasses exist — but subclasses re-declare the same fields anyway

AbstractWalletAmountTable
(@MappedSuperclass, lives in the dto/ package)

- totalCash, availableBalance, usedCash, lastUpdatedAmount, lastUpdatedDate, lastUpdatedTime
- Extended by all 16 *Amount entities (RetailerPrepaidAmount, DistributorGullakAmount, ...)

⚠ Every subclass re-declares the same fields

e.g. RetailerPrepaidAmount extends AbstractWalletAmountTable, then re-declares totalCash / usedCash / availableBalance / lastUpdatedAmount / lastUpdatedDate / lastUpdatedTime again with its own @Column mappings, plus extras (cutOfAmount, remark)

→ the shared base class exists but isn't actually relied on

Table/column name resolution — CommonMethods (fixed switch, not request input)

getEntityName(hierarchyType) — switch → "admin" / "enterprises" / "white_label" / "super_distributor" / "partner" / "distributor" / "retailer" / "retailer_user"

↓

getAmountTable() / getTransactionTable(type, walletType) → "{prefix}_{wallet}_amount" / "{prefix}_{wallet}_transaction_details"

✓ Explicit code comment on the native-SQL call site:
"amountTable and columnName must only come from trusted internal mappings — never accept from request"

Sub-users have no wallet of their own — resolved to their parent's wallet before every read:

- RU → parent Retailer's wallet (RT)
- AU → parent Admin's wallet (AD)
- WLAU → parent Whitelabel's wallet (WL)

Balance update path — plain read, compute in Java, plain write. No locking anywhere.

getCurrentBalance() — native SELECT available_balance — no lock → **Java computes** — newBalance = current ± amount → **DynamicWalletRepository.updateWallet()** — plain UPDATE — no WHERE balance check, no version col

⚠ Classic read-then-write race — confirmed no SELECT…FOR UPDATE / @Lock / version column anywhere in this path

FundingServiceImpl bulk upload: Executors.newFixedThreadPool(...) + CompletableFuture.supplyAsync(...) run this exact read-compute-write sequence concurrently across many rows — the race is not just theoretical, it's actively parallelized

⚠ CRITICAL — confirmed bug: multi-level reversal corrupts non-retailer wallets

TransactionEngineServiceImpl — manual transaction status flip from SUCCESS to FAILURE

1. wallet = getWalletByType(walletType, retailerId) — fetched **ONCE**, for the retailer only, before any loop
2. Loop runs over all 7 levels: retailer, distributor, partner, super_distributor, white_label, enterprises, admin
3. For every level's transaction record: **currentBalance = wallet.getAvailableBalance()** — still the retailer's snapshot from step 1, never re-fetched per level
4. newBalance is computed from that wrong base and written **directly** into that level's real amount table via dynamicWalletRepository.updateWallet() — a blind UPDATE, no arithmetic done in SQL

Net effect: 6 of the 7 levels in a multi-level commission reversal get their wallet balance overwritten using the RETAILER's balance as the arithmetic base — not their own. This silently corrupts distributor / partner / super-distributor / white-label / enterprises / admin wallet balances on every SUCCESS→FAILURE flip.

DynamicTransactionRepositoryImpl.getBalance() / .updateBalance() — dead code, wrong columns

References transaction_id / balance / hierarchy_id — columns that don't exist on the real amount tables (which use retailer_id / available_balance etc.). Verified unused anywhere in the codebase — would fail immediately if ever called.

Standard structure · Good practice, verified · Flagged risk / concurrency gap · Critical — confirmed correctness bug